Com S // Cpr E // Math 5250

Numerical Analysis of High-Performance Computing

Instructor: Songting Luo

Lecture 14: More on Finding Eigenvalues and Locality in Computer Memory

Outline

1. More on Finding Eigenvalues
2. Locality in Computer Memory

More on Finding Eigenvalues

Matrix Struct: header file (struct + macros)

$ISUHPC/lectures/lecture14/codes/matrix/matrix.h:

```
#ifndef __MATRIX_H__
#define __MATRIX_H__

typedef struct matrix matrix;
struct matrix
{ int rows; int cols; double* val; };

typedef struct vector vector;
struct vector
{ int size; double* val; };

// Shortcut evaluate functions
#define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
#define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
#define vget(vec,i) vec.val[(i-1)]
#define vgetp(vec,i) vec->val[(i-1)]
#define print_matrix(mat) print_matrix_full(mat,#mat);
#define print_vector(vec) print_vector_full(vec,#vec);
#define print_scalar(z) print_scalar_full(z,#z);
```

Matrix Struct: header file (declarations)

$ISUHPC/lectures/lecture14/codes/matrix/matrix.h:

```
// Function declarations
matrix new_matrix(const int rows, const int cols);
void print_matrix_full(const matrix* mat, char* varname);
matrix matrix_add(const matrix* A, const matrix* B);
matrix matrix_sub(const matrix* A, const matrix* B);
matrix matrix_mult(const matrix* A, const matrix* B);
matrix matrix_mult_v2(const matrix* A, const matrix* B);
matrix matrix_dot_mult(const matrix* A, const matrix* B);
//
vector new_vector(const int size);
void print_vector_full(const vector* vec, char* varname);
vector vector_add(const vector* x, const vector* y);
vector vector_sub(const vector* x, const vector* y);
double vector_dot_mult(const vector* x, const vector* y);
//
void print_scalar_full(const double* z, char* varname);
//
vector matrix_vector_mult(const matrix* A, const vector* x);
//
vector solve(const matrix* A, const vector* b);

#endif
```

Matrix Struct: main function

$ISUHPC/lectures/lecture14/codes/matrix/old_main.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include "matrix.h"

int main()
{
    // Matrices
    matrix A = new_matrix(5,5);
    matrix B = new_matrix(5,5);

    for(int i=1; i<=5; i++ )
        for (int j=1; j<=5; j++ )
        {
            mget(A,i,j) = -1.0*(i==j)
                + 2.0*(i-1==j) + 2.0*(j-1==i);
            mget(B,i,j) =  2.0*(i==j)
                + 1.0*(i-1==j) + 1.0*(j-1==i);
        }

    // Print matrices
    print_matrix(&A);
    print_matrix(&B);
```

Matrix Struct: main function

$ISUHPC/lectures/lecture14/codes/matrix/old_main.c:

```
    // Add/Subtract/Multiply matrices
    matrix  Csum = matrix_add(&A,&B); print_matrix(&Csum);
    matrix Cdiff = matrix_sub(&A,&B); print_matrix(&Cdiff);
    matrix Cprod = matrix_mult(&A,&B); print_matrix(&Cprod);
    matrix  Cdot = matrix_dot_mult(&A,&B); print_matrix(&Cdot);

    // Vectors
    vector x = new_vector(5);
    vector y = new_vector(5);

    vget(x,1) = 1.0;  vget(y,1) = 1.0;
    vget(x,2) = 0.0;  vget(y,2) = 2.0;
    vget(x,3) = 1.0;  vget(y,3) = 3.0;
    vget(x,4) = 0.0;  vget(y,4) = 4.0;
    vget(x,5) = 1.0;  vget(y,5) = 5.0;

    // Print vectors
    print_vector(&x);
    print_vector(&y);
```

Matrix Struct: main function

$ISUHPC/lectures/lecture14/codes/matrix/old_main.c:

```

    // Add/Subtract/Multiply vectors
    vector  zsum = vector_add(&x,&y); print_vector(&zsum);
    vector zdiff = vector_sub(&x,&y); print_vector(&zdiff);
    double  zdot = vector_dot_mult(&x,&y); print_scalar(&zdot);

    // Matrix vector multiply
    vector Ax = matrix_vector_mult(&A,&x);
    print_vector(&Ax);

    // Linear solve via Gaussian elimination
    vector soln = solve(&A,&y);
    print_vector(&soln);
}
```

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric marix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}^n} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Solution:

$$\|A\mathbf{x} - \alpha\mathbf{x}\|^2 = (A\mathbf{x} - \alpha\mathbf{x})^T (A\mathbf{x} - \alpha\mathbf{x}) = \left(\mathbf{x}^T A - \alpha\mathbf{x}^T\right)(A\mathbf{x} - \alpha\mathbf{x})$$
$$= \mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T\mathbf{x}$$

(concave up quadratic function, unique global min where $d/d\alpha$ is zero)

$$\frac{d}{d\alpha}\left(\mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T\mathbf{x}\right) = 0 \qquad \Longrightarrow \qquad 2\left(\alpha\mathbf{x}^T\mathbf{x} - \mathbf{x}^T A\mathbf{x}\right) = 0$$

$$\therefore \quad \hat{\alpha} = R_A(\mathbf{x}) = \frac{\mathbf{x}^T A\mathbf{x}}{\mathbf{x}^T\mathbf{x}} \qquad \text{(Rayleigh Quotient)}$$

If $\mathbf{v}$ is an eigenvector of A, then

$$R_A(\mathbf{v}) = \frac{\mathbf{v}^T A\mathbf{v}}{\mathbf{v}^T\mathbf{v}} = \frac{\mathbf{v}^T \lambda\mathbf{v}}{\mathbf{v}^T\mathbf{v}} = \lambda\left(\frac{\mathbf{v}^T\mathbf{v}}{\mathbf{v}^T\mathbf{v}}\right) = \lambda.$$

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \cdots > |\lambda_n|$$

Guess and eigenvector:

$$\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, \mathbf{v}_j$$

where $\mathbf{v}_j$ for $j = 1, 2, \ldots, n$ are the n linearly independent eigenvectors of A.

How do we improve this guess?

$$\mathbf{w}^{(1)} = A\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, A\mathbf{v}_j = \sum_{j=1}^{n} \alpha_j \, \lambda_j \mathbf{v}_j$$

$$\mathbf{w}^{(2)} = A\mathbf{w}^{(1)} = A^2\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, A^2\mathbf{v}_j = \sum_{j=1}^{n} \alpha_j \, \lambda_j^2 \mathbf{v}_j$$

$$\mathbf{w}^{(k)} = A\mathbf{w}^{(k-1)} = A^k\mathbf{w}^{(0)} = \sum_{j=1}^{n} \alpha_j \, A^k\mathbf{v}_j = \sum_{j=1}^{n} \alpha_j \, \lambda_j^k \mathbf{v}_j \approx \alpha_1 \, \lambda_1^k \mathbf{v}_1$$

$$\therefore \quad \lambda_1 \approx R_A\left(\mathbf{w}^{(k)}\right)$$

Power Iteration Algorithm

INPUT: $\mathbf{v}^{(0)} \in \mathbb{R}^n$, `TOL`, `MaxIters`, $A \in \mathbb{R}^{n \times n}$ (real & symmetric)

OUTPUT: $\lambda^{(k)}$ (approximation to λ_1 after k iterations)

$\mathbf{v}^{(0)} = \frac{\mathbf{v}^{(0)}}{\|\mathbf{v}^{(0)}\|}$ and $\lambda^{(0)} = \left[\mathbf{v}^{(0)}\right]^T A\mathbf{v}^{(0)}$

`mstop = 0` and $k = 0$

while `mstop==0` **do**
  $k = k + 1$

  $\mathbf{w} = A\mathbf{v}^{(k-1)}$, $\mathbf{v}^{(k)} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$, and $\lambda^{(k)} = \left[\mathbf{v}^{(k)}\right]^T A\mathbf{v}^{(k)}$

  if $|\lambda^{(k)} - \lambda^{(k-1)}| < \texttt{TOL}$ **or** $k == \texttt{MaxIters}$ **then**
    $\texttt{mstop} = 1$
  end if
end while

Shifted Inverse Iteration

1. Apply the power iteration method to A^{-1}

   NOTE: A^{-1} has eigenvalues
   $$\frac{1}{\lambda_j}$$
   and the same eigenvectors as A. Therefore, power iteration method will give an approximation to λ_n^{-1}.

2. Apply the power iteration method to
   $$(A - \mu I)^{-1}.$$
   NOTE: $(A - \mu I)^{-1}$ has eigenvalues
   $$(\lambda_j - \mu)^{-1}$$
   and the same eigenvectors as A.

   Therefore, power iteration method will give an approximation to $(\lambda_m - \mu)^{-1}$ where m is the index such that
   $$|\lambda_m - \mu| = \min_{1 \le j \le n} |\lambda_j - \mu|$$

Key improvement: the closer μ is to the eigenvalue of interest, λ_k, the faster this method will converge.

Shifted Inverse Power Iteration Algorithm

INPUT: $\mu \in \mathbb{R}$, $\mathbf{v}^{(0)} \in \mathbb{R}^n$, $\texttt{TOL}$, $\texttt{MaxIters}$, $A \in \mathbb{R}^{n \times n}$ (real & symmetric)

OUTPUT: $\lambda^{(k)}$ (approx to the closest eigenvalue to μ, λ_m, after k iterations)

$\mathbf{v}^{(0)} = \frac{\mathbf{v}^{(0)}}{\|\mathbf{v}^{(0)}\|}$ and $\lambda^{(0)} = \left[\mathbf{v}^{(0)}\right]^T A\mathbf{v}^{(0)}$

$\texttt{mstop} = \texttt{0}$ and $k = 0$

while $\texttt{mstop==0}$ **do**
  $k = k + 1$

  Solve $(A - \mu I)\,\mathbf{w} = \mathbf{v}^{(k-1)}$, $\mathbf{v}^{(k)} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$, and $\lambda^{(k)} = \left[\mathbf{v}^{(k)}\right]^T A\mathbf{v}^{(k)}$

  if $|\lambda^{(k)} - \lambda^{(k-1)}| < \texttt{TOL}$ **or** $k == \texttt{MaxIters}$ **then**
    $\texttt{mstop} = 1$
  end if
end while

Rayleigh Quotient Iteration

Idea: Instead of fixing a value of μ, update the value of μ as we iterate using the Rayleigh quotient.

Rayleigh Quotient Iteration

Idea: Instead of fixing a value of μ, update the value of μ as we iterate using the Rayleigh quotient.

INPUT: $\mathbf{v}^{(0)} \in \mathbb{R}^n$, `TOL`, `MaxIters`, $A \in \mathbb{R}^{n\times n}$ (real & symmetric)

OUTPUT: $\lambda^{(k)}$ (approx to some eigenvalue of A after k iterations)

$\mathbf{v}^{(0)} = \frac{\mathbf{v}^{(0)}}{\|\mathbf{v}^{(0)}\|}$ and $\lambda^{(0)} = \left[\mathbf{v}^{(0)}\right]^T A\mathbf{v}^{(0)}$

`mstop = 0` and $k = 0$

while `mstop==0` **do**
  $k = k + 1$

  Solve $\left(A - \lambda^{(k-1)} I\right)\mathbf{w} = \mathbf{v}^{(k-1)}$, $\mathbf{v}^{(k)} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$, $\lambda^{(k)} = \left[\mathbf{v}^{(k)}\right]^T A\mathbf{v}^{(k)}$

  if $|\lambda^{(k)} - \lambda^{(k-1)}| < \texttt{TOL}$ **or** $k == \texttt{MaxIters}$ **then**
    $\texttt{mstop} = 1$
  end if
end while

Convergence Rates

Power iteration

Suppose that $|\lambda^{(1)}| > |\lambda^{(2)}| \geq |\lambda^{(3)}| \geq \ldots \geq |\lambda^{(N)}|$ and that $\mathbf{v}_1 \cdot \mathbf{v}^{(0)} \neq 0$. Then the power iteration after k steps produces an approximate eigenvector and eigenvalue that satisfies:

$$\|\mathbf{v}^{(k)} - \mathbf{v}_1\| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right) \qquad \text{and} \qquad \left|\lambda^{(k)} - \lambda_1\right| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right).$$

Another way to say this is the following:

$$\lim_{k\to\infty} \frac{\left|\lambda^{(k+1)} - \lambda_1\right|}{|\lambda^{(k)} - \lambda_1|} = c\left|\frac{\lambda_2}{\lambda_1}\right|^2.$$

Convergence Rates

Shifted inverse power iteration

Suppose that λ_J is the eigenvalue closest to μ and λ_K is the next closest:

$$|\lambda_J - \mu| < |\lambda_K - \mu| \leq |\lambda_i - \mu| \quad \forall \quad i \neq J.$$

Assume $\mathbf{v}_J \cdot \mathbf{v}^{(0)} \neq 0$. The shifted inverse iteration produces at step k the following approximations:

$$\|\mathbf{v}^{(k)} - \mathbf{v}_J\| = \mathcal{O}\left(\left|\frac{\lambda_J - \mu}{\lambda_K - \mu}\right|^k\right) \qquad \text{and} \qquad \left|\lambda^{(k)} - \lambda_J\right| = \mathcal{O}\left(\left|\frac{\lambda_J - \mu}{\lambda_K - \mu}\right|^{2k}\right).$$

Another way to say this is the following:

$$\lim_{k\to\infty} \frac{\left|\lambda^{(k+1)} - \lambda_J\right|}{|\lambda^{(k)} - \lambda_J|} = c\left|\frac{\lambda_J - \mu}{\lambda_K - \mu}\right|^2.$$

Convergence Rates

Rayleigh quotient iteration

If $\mathbf{v}^{(0)}$ is sufficiently close to the eigenvector $\mathbf{v}_J$ of A the then the Rayleigh quotient iteration produces approximations that satisfy:

$$\|\mathbf{v}^{(k+1)} - \mathbf{v}_J\| = \mathcal{O}\left(\|\mathbf{v}^{(k)} - \mathbf{v}_J\|^3\right) \qquad \text{and} \qquad \left|\lambda^{(k+1)} - \lambda_1\right| = \mathcal{O}\left(\left|\lambda^{(k)} - \lambda_J\right|^3\right).$$

Another way to say this is the following:

$$\lim_{k\to\infty} \frac{\left|\lambda^{(k+1)} - \lambda_J\right|}{\left|\lambda^{(k)} - \lambda_J\right|^3} = c.$$

Convergence Rates

Rayleigh quotient iteration

If $\mathbf{v}^{(0)}$ is sufficiently close to the eigenvector $\mathbf{v}_J$ of A the then the Rayleigh quotient iteration produces approximations that satisfy:

$$\|\mathbf{v}^{(k+1)} - \mathbf{v}_J\| = \mathcal{O}\left(\|\mathbf{v}^{(k)} - \mathbf{v}_J\|^3\right) \qquad \text{and} \qquad \left|\lambda^{(k+1)} - \lambda_1\right| = \mathcal{O}\left(\left|\lambda^{(k)} - \lambda_J\right|^3\right).$$

Another way to say this is the following:

$$\lim_{k\to\infty} \frac{\left|\lambda^{(k+1)} - \lambda_J\right|}{\left|\lambda^{(k)} - \lambda_J\right|^3} = c.$$

Next Lecture: a method for computing all the eigenvalues of a real symmetric matrix in one fell swoop.

Locality in Computer Memory

How fast are computers?

Kilo	=	thousand $\left(10^3\right)$
Mega	=	million $\left(10^6\right)$
Giga	=	billion $\left(10^9\right)$
Tera	=	trillion $\left(10^{12}\right)$
Peta	=	10^{15}
Exa	=	10^{18}

Processor speeds usually measured in Gigahertz these days.

Hertz means *machine cycles per second*.

One operation may take a few cycles.

So a 1 GHz processor (10^9 cycles per second) can do
$> 100,000,000$ floating point operations per second ($>$ 100 Megaflop/s).

Modern supercomputers

High-Performance Computing (HPC) often means heavy-duty computing on clusters or supercomputers with hundreds of thousands of cores.

`http://top500.org`: top 500 fastest supercomputers in the world

How fast are computers?

Not long ago counting **flops** was the best way to measure performance for scientific computing.

Example: Computing matrix-matrix product $C = AB$.

If A and B are $N \times N$, then so is C.

Each element c_{ij} is the inner product of

1. i^{th} row of A
2. j^{th} column of B

Requires N multiplications and $N - 1$ additions to compute c_{ij}.

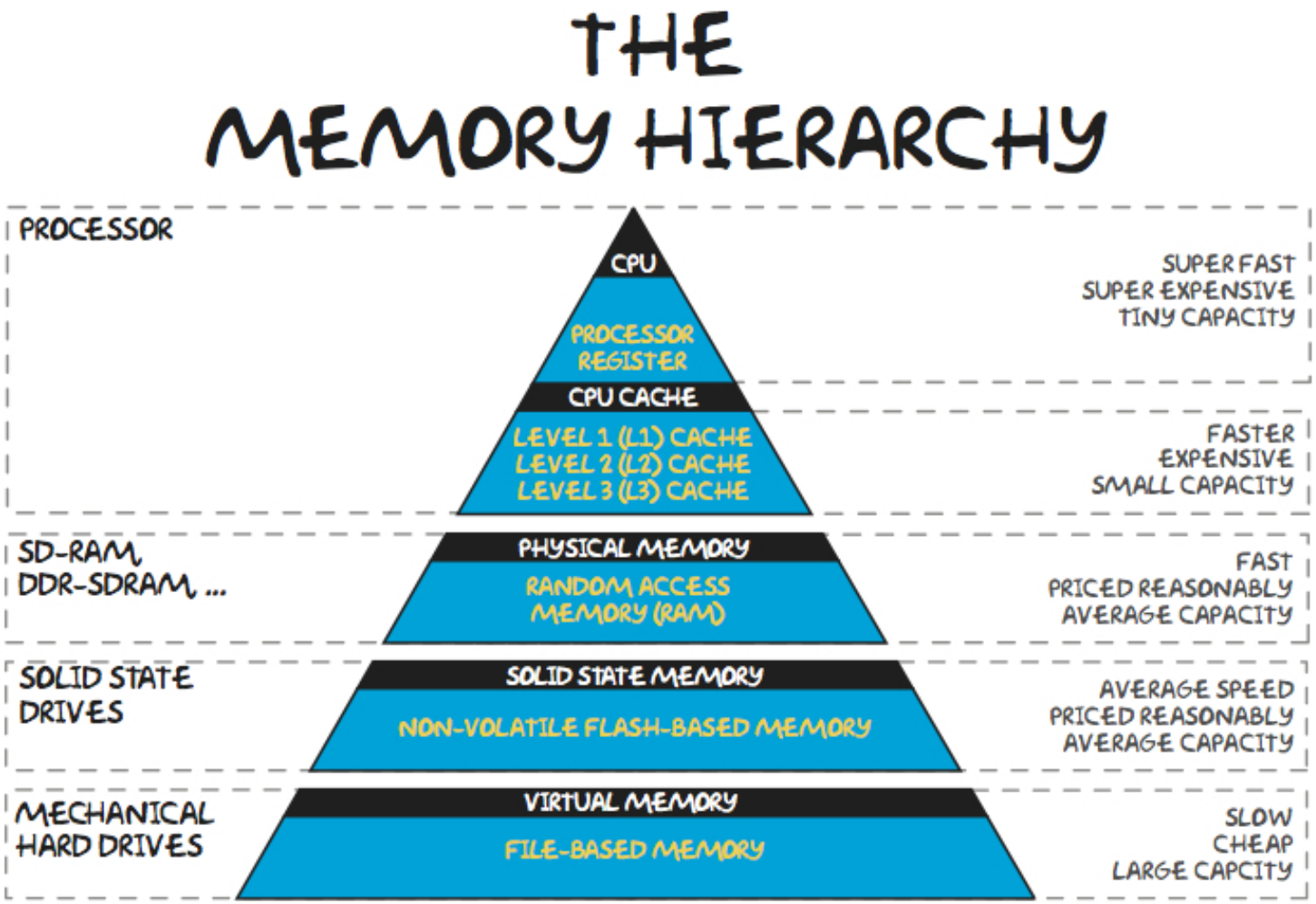
N^2 elements in $C \implies$ requires $\mathcal{O}(N^3)$ floating point operations.

If $N = 10,000 = 10^4 \implies N^3 = 10^{12}$
($>$ 1000 seconds on 1 GHz processor)

But these days, the bottle neck is often getting data to and from the processor!

Note: each element of A and B is used N times.

Memory hierarchy



Source: `http://computerscience.chemeketa.edu/`

Memory hierarchy

CPU Registers

A register is a memory location within the CPU itself, which means it provides very fast access to the information it contains. Only the most immediate set of information (current instruction and a few pieces of data) is in the registers.

CPU Cache

This is special memory that is smaller and faster than the normal main memory (RAM) and that is housed closer to the CPU. By maintaining a copy of parts of what is in main memory (RAM), it allows quicker access to some information. As far as the logic of a program goes, the cache doesn't really exist; we always request information from main memory (RAM). The cache just makes it so that sometimes those memory requests come back extra fast.

Source: `http://computerscience.chemeketa.edu/`

Memory hierarchy

RAM - Main Memory

It generally runs significantly slower than the CPU - each time the CPU has to access main memory, it can represent a significant slow down.

Secondary Memory

Main memory needs power to persist. Secondary memory is the general term for things like solid state drives, hard drives, CD drives, flash drives, etc... that can store information long term. Not only can they retain information without power, they also allow more information to be stored in less space and with less cost. Unfortunately, these technologies are all slower than main memory. Information stored in secondary memory must be loaded into main memory (RAM) before the processor can access it.

Source: `http://computerscience.chemeketa.edu/`

32-bit vs. 64-bit architecture

Each byte in memory has an address, which is an integer. On 32-bit machines, registers can only store

$$2^{32} = 4294967296 \approx 4\,\text{billion distinct addresses}$$

$\Longrightarrow$ at most 4 GB of memory can be adressed.

Newer machines often have more, leading to the need for 64-bit architectures (8 bytes for addresses).

$$2^{64} = 1.844674407370955 \times 10^{19}\,\text{distinct addresses}$$

$\Longrightarrow$ could address an exabyte of memory.

CPU and registers

CPU – central processor unit

Executes instructions such as add or multiply.

Takes data from registers, performs operations, stores back to registers.

Transferring between registers and processor is very fast.

Different types of registers:

- integer, floating point
- instruction registers
- address registers

Generally a very small number of registers.

Data and instructions must be transferred between other memory and registers as needed.

Memory hierarchy

Between registers and memory there are 2 or 3 levels of cache, each larger but slower.

Different access times:

Registers:	1 cycle
L1 cache:	a few cycles
L2 cache:	$\sim$ 10 cycles
(Main) Memory:	$\sim$ 250 cycles
Hard drive:	$\sim$ 1000s of cycles

Terminology

Latency: the period of time that one component in a system is spinning its wheels waiting for another component. Latency, therefore, is wasted time.

Throughput: the amount of work that can be completed per unit time.

Exploit parallelism to hide latency and increase throughput.

Even a *single core* machine has lots of things going on at once.

For example:

- Pipelined operations
- Executing/fetching/storing
- Prefetching future instructions
- Prefetching data into cache

Reducing memory latency

Reduce memory fetches by reusing data in cache as much as possible.
Requires temporal locality.

Very simple Python example: if `len(x)` much larger than cache size:

```
z = 0.0;  w = 0.0;
for i in range(len(x)):
    z = z + x[i]
for i in range(len(x)):
    w = w + 3.0*x[i]
```

should be rewritten as

```
for i in range(len(x)):
    z = z + x[i]
    w = w + 3.0*x[i]
```

Note: both are bad in Python, use `z = np.sum(x); w = 3*z;`

Cache lines

When data is brought into cache, more than 1 value is fetched at a time.

A cache line typically holds 64 or 128 consecutive bytes (8 or 16 floats).

L1 Cache might hold 1000 cache lines.

Cache miss occurs if the the value you need next is not in cache.

Another cache line will be brought from higher up the hierachy, and may displace some variables in cache.

Those cache lines will first have to be written back to memory.

Bottom line: Good to do lots of work on each set of data while in cache, before it has to be written back.

Organize algorithm for **Temporal locality.**

Spatial locality

Also good to organize algorithm so data that is **consecutive in memory** is used together when possible.

If data you need is scattered through memory, many cache lines will be needed and will contain data you don't need.

This is called **spatial locality.**

Spatial locality: example #1

$ISUHPC/lectures/lecture14/codes/matrix/example1.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include <assert.h>
#include <time.h>
#include "matrix.h"

int main()
{
    // Get dimensions
    int NumRows;
    int NumCols;
    printf("    Input number of rows: ");
    scanf("%i",&NumRows);
    printf(" Input number of columns: ");
    scanf("%i",&NumCols);
    assert(NumRows>0); assert(NumCols>0);

    // Matrices
    matrix A = new_matrix(NumRows,NumCols);
```

Spatial locality: example #1

$ISUHPC/lectures/lecture14/codes/matrix/example1.c:

```
    clock_t time0,time1,time2;

    // Matrix set
    time0 = clock();
    for (int i=1; i<=NumRows; i++)
       for (int j=1; j<=NumCols; j++)
       { mget(A,i,j) = 1.0; }
    time1 = clock();
    for (int j=1; j<=NumCols; j++)
       for (int i=1; i<=NumRows; i++)
       { mget(A,i,j) = 1.0; }
    time2 = clock();

    // Report times
    double cpu_time1 = ((double)(time1-time0))
       /((double)(CLOCKS_PER_SEC));
    double cpu_time2 = ((double)(time2-time1))
       /((double)(CLOCKS_PER_SEC));
    printf(" cpu_time1 = %11.5e\n",cpu_time1);
    printf(" cpu_time2 = %11.5e\n",cpu_time2);
 }
```

Spatial locality: example #1

```
$ make example1
gcc -O3 -Wall -std=c99 -c example1.c
gcc  matrix.o  example1.o -o ex1.exe

$ ex1.exe
    Input number of rows: 1000
 Input number of columns: 1000
 cpu_time1 = 6.78000e-04
 cpu_time2 = 7.88500e-03

$ ex1.exe
    Input number of rows: 10000
 Input number of columns: 10000
 cpu_time1 = 1.19541e-01
 cpu_time2 = 1.38493e+00
```

Spatial locality: example #2

$ISUHPC/lectures/lecture14/codes/matrix/example2.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include <assert.h>
#include <time.h>
#include "matrix.h"

int main()
{
    // Get dimensions
    int NumRows;
    int NumCols;
    printf("    Input number of rows: ");
    scanf("%i",&NumRows);
    printf(" Input number of columns: ");
    scanf("%i",&NumCols);
    assert(NumRows>0); assert(NumCols>0);

    // Matrices
    srand((unsigned)time(NULL));
    matrix A = new_matrix(NumRows,NumCols);
```

Spatial locality: example #2

$ISUHPC/lectures/lecture14/codes/matrix/example2.c:

```
        for (int j=1; j<=A.cols; j++)
        { mget(A,i,j) = ((double)rand()/(double)RAND_MAX); }

    // Clock times
    clock_t time0,time1,time2;

    // Matrix multiply
    time0 = clock();
    matrix_mult(&A,&A);
    time1 = clock();
    matrix_mult_v2(&A,&A);
    time2 = clock();

    // Report times
    double cpu_time1 = ((double)(time1-time0))
        /((double)(CLOCKS_PER_SEC));
    double cpu_time2 = ((double)(time2-time1))
        /((double)(CLOCKS_PER_SEC));
    printf(" cpu_time1 = %11.5e\n",cpu_time1);
    printf(" cpu_time2 = %11.5e\n",cpu_time2);
}
```

Spatial locality: example #2

$ISUHPC/lectures/lecture14/codes/matrix/matrix.c:

```
matrix matrix_mult(const matrix* A, const matrix* B)
{
   const int rowsA = A->rows; const int colsA = A->cols;
   const int rowsB = B->rows; const int colsB = B->cols;
   assert(colsA==rowsB);
   matrix C = new_matrix(rowsA,colsB);

   for (int i=1; i<=rowsA; i++)
      for (int j=1; j<=colsB; j++)
         for (int k=1; k<=colsA; k++)
         {
            mget(C,i,j) += mgetp(A,i,k)*mgetp(B,k,j);
         }

   return C;
}
```

Spatial locality: example #2

$ISUHPC/lectures/lecture14/codes/matrix/matrix.c:

```
matrix matrix_mult_v2(const matrix* A, const matrix* B)
{
   const int rowsA = A->rows; const int colsA = A->cols;
   const int rowsB = B->rows; const int colsB = B->cols;
   assert(colsA==rowsB); matrix C = new_matrix(rowsA,colsB);

   matrix Btranspose = new_matrix(rowsB,colsB);
   for (int i=1; i<=rowsA; i++)
      for (int j=1; j<=colsB; j++)
      { mget(Btranspose,i,j) = mgetp(B,j,i); }

   for (int i=1; i<=rowsA; i++)
      for (int j=1; j<=colsB; j++)
         for (int k=1; k<=colsA; k++)
         { mget(C,i,j) += mgetp(A,i,k)*mget(Btranspose,j,k); }

   return C;
}
```

Spatial locality: example #2

```
$ make example2
gcc -O3 -Wall -std=c99 -c example2.c
gcc  matrix.o  example2.o -o ex2.exe

$ ex2.exe
    Input number of rows: 1000
 Input number of columns: 1000
  cpu_time1 = 7.43073e+00
  cpu_time2 = 1.03023e+00
```

Lab assignment: eigenvalue solvers